SolvedBooks.pk Presents

Computer Science

Class 10

Easy Notes

(Complete Notes of the Textbook)

Sindh Textbook Board • English Medium

Unit 1

Problem Solving and Algorithm Designing

Unit 2

Basics of Programming in C++

Unit 3

Input/Output Handling in C++

Unit 4

Control Structure

Unit 5

Functions

Unit 6

Digital Logic and Design

Unit 7

Introduction to Scratch

Easy Notes of All 28 Lessons

Simple notes • Key points • Easy to revise

SolvedBooks.pk

solvedbooks.pk | WhatsApp: +92 301 8981055

Table of Contents

Tap or click any lesson to jump straight to it.

Unit 1 : Problem Solving and Algorithm Designing 4

- Problem Solving and Its Steps 4
- Algorithms: Advantages and Writing Algorithms 4
- Flowcharts: Symbols, Advantages and Drawing Flowcharts 4
- Data Structures: Array, Stack and Queue 5
- Tree and Graph Data Structures 5

Unit 2 : Basics of Programming in C++ 7

- Computer Programs and Programming Languages 7
- Translators, IDE and Program Errors 7
- Variables and Data Types 7

Unit 3 : Input/Output Handling in C++ 9

- Input and Output Streams, Header Files and Input Functions 9
- Comments and Escape Sequences 9
- Operators in C++ 9
- Programs: Formulas, Marksheet and Finding Errors 10

Unit 4 : Control Structure 11

- Selection: if, else if and switch 11
- Loops: for, while and do-while 11
- Jump Statements and Blocks of Statements 12
- Programs with Selection and Loops 12

Unit 5 : Functions 14

- Functions: Built-in and User-defined; Declaration and Definition 14
- Header Files, Built-in Functions and Variable Scope 14
- Passing Arguments and Returning Values 15
- Programs Using Functions 15

Unit 6 : Digital Logic and Design 17

- Basic Logic Gates: AND, OR and NOT 17
- Universal Gates: NAND and NOR 17
- Boolean Algebra and Truth Tables 17
- Simplifying Expressions and Designing Circuits 18

Unit 7 : Introduction to Scratch 19

- Scratch Basics: Sprites, Scripts and Backdrops 19

The Stage, Coordinates and Block Palettes	19
Motion and Control Blocks	20
Scratch Online and Offline; Sample Programs	20

Problem Solving and Algorithm Designing

UNIT 1

Problem Solving and Its Steps

1. What is a problem?

A problem is a task that has to be solved. For example, "**to bake a cake**" is a problem — baking a cake is a task / problem to be solved, and the steps for doing it are its algorithm.

2. The first step of problem solving

To find a feasible solution to a problem, the first step is to **identify and analyze the problem** — in the problem-solving process, the first step is defining / identifying the problem.

3. The four steps of problem solving

- **1. Define the problem:** identifying and describing the real problem.
- **2. Generate alternative solutions:** making a list of possible solutions.
- **3. Evaluate and select an alternative:** choosing the best solution.
- **4. Implement and follow up on the solution:** measuring the results and seeing whether the solution worked.

Algorithms: Advantages and Writing Algorithms

1. What is an algorithm?

An **algorithm** is the step by step solution of a problem in simple language. It is a set of instructions / steps / rules for solving a problem.

2. Advantages of developing algorithms

- An algorithm gives the best possible answer every time.
- It helps in work needing accuracy and in solving similar problems again and again.
- Computer programs can be built with the help of algorithms.
- In a good algorithm the input and output are precise, every step is clear, and it can be used in any programming language.

3. Algorithm: find the greater of two numbers

Step 1: Start

Step 2: Declare variables num1, num2 and greater

Step 3: Read values num1 and num2

Step 4: If num1 is greater than num2 then greater = num1, else greater = num2

Step 5: Display greater

Step 6: Stop

4. Algorithm: area of a triangle

Step 1: Start

Step 2: Declare variables base, height and area

Step 3: Read values base and height

Step 4: Apply formula {Area = (base x height) / 2}, i.e. area = (base x height) / 2

Step 5: Display area

Step 6: Stop

5. Ordering the steps: baking a cake

The steps of the cake-baking algorithm in correct order: Start; Preheat the oven; Gather the ingredients; Mix together the ingredients to make the batter; Grease a pan; Pour the batter into the pan; Put the pan in the oven; Set a timer; When the timer goes off, take the pan out of the oven; End.

Flowcharts: Symbols, Advantages and Drawing Flowcharts

1. What is a flowchart?

A **flowchart** shows the logic of a program graphically. It is the pictorial / graphical representation of the problem-solving process.

2. Flowchart symbols

Symbol	Used for
Oval	Start and Stop
Parallelogram	Input / Output
Rectangle	Processing — calculation steps
Diamond	Decision — where a choice is needed

3. Advantages of designing flowcharts

- Flowcharts communicate the logic of a program to others and make it easy to understand.
- They are useful program documents with which a program can be quickly understood or its logic changed.
- They work as a guide / blueprint while coding a program.
- They show the operations of solving a problem and their sequence visually before writing it.

4. Flowchart: gross salary

Problem: calculate gross salary by adding 20% house rent and 30% medical allowances in basic salary. Flowchart: Start (oval) — Input basic salary (parallelogram) — House Rent = Basic Salary x 20 / 100 and Medical Allowance = Basic Salary x 30 / 100 (rectangle) — Gross Salary = Basic Salary + House Rent + Medical Allowance (rectangle) — Display Gross Salary (parallelogram) — Stop (oval). So Gross Salary = 150% of Basic Salary.

5. Drawing flowcharts for all the algorithms of this unit

The flowchart of every algorithm follows the same pattern: Start (oval) — Input (parallelogram) — calculation steps (rectangle) — Display result (parallelogram) — Stop (oval); where a choice is needed, the decision diamond is used. Making tea, sum of two numbers, average of three numbers, volume of a box, percentage, greater number, area of a triangle and gross salary — each flowchart is made by turning that algorithm's steps into these shapes; the model in the unit (the sum-and-percentage flowchart with the percent > 70 decision) is an example of it.

Data Structures: Array, Stack and Queue

1. Linear and non-linear data structures

A **graph** is a non-linear data structure; its elements are not connected in sequence. Array, queue and stack are linear data structures.

2. Array and the need of an index

Every element of an array is identified by an **index**; indexes are a set of consecutive numbers with which any element can be directly found and used. Without an index, it would not be known which element is stored where. **Traversing** an array means accessing each and every element of the array.

3. Stack: push, pop and overflow

When data is **pushed** in a stack, it means data is inserted. In the sketch of a stack: with Push a new element enters from the top and with Pop the top element is removed; when the stack is completely full it is in the **Overflow** state and nothing can be added to it.

4. Queue: enqueue and dequeue

In the sketch of a queue: with **Enqueue** a new element enters at the rear end and with **Dequeue** an element is removed from the front end.

5. Stack vs queue

Stack	Queue
Data is added or removed only from the top	A new element enters at the rear end and the old element is removed from the front end
Works in LIFO / FILO order (last in, first out)	Works in FIFO order (first in, first out)
Operations are push and pop	Operations are enqueue and dequeue

Tree and Graph Data Structures

1. Tree

A **tree** is a non-linear data structure that stores data in a hierarchical manner; it shows the hierarchical relationship between elements. In a **binary tree**, each node can have a maximum of two nodes (children).

2. Graph

A **graph** has nodes / vertices and edges joining them. It has no root or child; each node can have many edges, and a cycle can also be formed.

3. Tree vs graph

Tree	Graph

Shows hierarchical data	Nodes / vertices are joined by edges
Has one root node; every node has one parent with children	No root or child; each node can have many edges
No cycle	A cycle can also be formed

4. Drawing a tree with six nodes and a graph with five nodes

(a) Tree: root node 1 at the top; its children 2 and 3; node 2's children 4 and 5; node 3's child 6 — six nodes in all, every edge from parent to child, no cycle.

(b) Graph: five nodes 1 to 5; edges can join any two nodes, for example 1-2, 2-3, 3-4, 4-5, 5-1 and 1-3 — a cycle can be formed.

Basics of Programming in C++

UNIT 2

Computer Programs and Programming Languages

1. What is a computer program?

A computer program is a collection / set of **instructions** that is understood by a computer to perform tasks. The person who writes programs is called a programmer, and specialized programs such as a compiler convert those instructions into machine-readable instructions.

2. Low-level and high-level languages

High-level languages have syntax that is **easily readable by humans** — they are closer to human languages and are written in English-like words. Low-level languages have syntax that is easily readable by machines and hard for humans.

3. Machine language vs assembly language

Machine language	Assembly language
At the lowest level; written in 0 and 1 (binary)	One level above machine language; written in simple English language, easy for humans to read, write and maintain
Executed directly by the computer; needs no translator	Needs an assembler to convert it into machine code
First-generation language	Second-generation language

4. Five common high-level languages

Languages named in the unit: **C++, Java, Python, Visual Basic and PHP**. C++ is a general-purpose high-level language developed as an enhancement of the C language to include the object-oriented concept and to make writing programs easier and more pleasant.

Translators, IDE and Program Errors

1. Compiler and interpreter

Compiler	Interpreter
Translates codes all at once — it translates the whole program in one session and creates an executable file	Translates codes line-by-line and reports the error as soon as it is found

2. Source code and object code

Source code is a program written in any language, as the programmer writes it. **Object / machine code** is the translated form that the computer runs; the translator converts the source code into the target machine language.

3. Integrated Development Environment (IDE)

An IDE facilitates a programmer to edit source code, complete and highlight syntaxes, and debug and compile codes — all of these in one environment.

4. Advantages of using an IDE

- An IDE gives compilation, linking, loading and debugging tools in a single environment.
- Code completion improves the programming workflow.
- It automatically checks for errors to ensure top-quality code.
- Refactoring allows comprehensive and mistake-free renaming.
- It also gives facilities such as syntax highlighting and autocomplete.

5. Program errors

All errors detected by users are typically **logical errors**: a program with a logical error keeps running but gives a wrong result, so the user notices it from the wrong result; syntax errors are caught by the compiler / IDE itself.

Variables and Data Types

1. Rules for naming a variable

Allowed names for declaring a variable can contain **letters, digits and underscores**. Names begin with a letter or an underscore, cannot contain whitespaces, cannot be one of the reserved words, and cannot be the same as a data type.

2. Ten meaningful and valid variable names

studentName, totalMarks, firstName, lastName, rollNo, _average, myVar1, age2, price, class10 — all begin with a letter or an underscore, contain letters, digits and underscores, have no whitespace, use no reserved word, and are all shorter than 32 characters.

3. Data types and memory size

Data type	Stores	Space in memory
char (character)	A single character	1 byte
int (integer)	Whole numbers	4 bytes
float (floating point)	Fractional numbers	4 bytes
double (double floating point)	Fractional numbers	8 bytes — the most space
bool	True or false — 0 (false) or 1 (true)	—

4. Strings in C++

Strings are variables that store non-numerical values longer than one single character. The C++ language library supports strings through the standard **string class**; a string is not a fundamental type but behaves in a similar way to fundamental types in its most basic use; it can be declared without an initial value and assigned a value during execution.

5. Declaring and initializing a variable

A **declaration** tells the compiler where and how much storage to create for the variable; it specifies a data type and a name: `data_type variable_name;` **Initialization** assigns an initial value in the declaration itself, with an equal sign: `data_type variable_name = value;` Variables do not need to be assigned initial values; a value can also be assigned later.

Input/Output Handling in C++

UNIT 3

Input and Output Streams, Header Files and Input Functions

1. The <iostream> header file

The C++ header file **<iostream>** contains the function prototypes / definitions for the standard Input and Output functions. I/O Stream is a standard library file that contains definitions of Standard Input and Output streams.

2. Input and output operators

- **Extraction operator (>>):** used for input stream — `cin` reads data from the standard input device with the extraction operator (>>).
- **Insertion operator (<<):** used for output stream with `cout`.

3. Input functions: gets(), getchar(), getch() and getche()

- **gets()** stands for **get string** — gets() reads a string of characters from the standard input and stores them until a newline character.
- **getchar()** needs pressing the **Enter key** from the keyboard after entering the character.
- **getch() and getche()** are included in the **<conio.h>** header file (Console Input and Output).

Comments and Escape Sequences

1. Types of comment statements in C++

Two types: **1. Single Line Comment** — with a double slash (//), for a one-line explanation; **2. Multi Line Comment** — with (/* and */), for explanations over many lines. Both are ignored by the compiler, meaning they are not executed.

2. Escape sequences \a and \r in a program

```
#include<iostream> using namespace std; int
main() { cout << "Beep\a"; cout << "\rStart";
return 0; }
```

- **\a** is Alert / alarm — it causes a beep sound in the computer.
- **\r** is Carriage Return — it positions the cursor at the beginning of the current line.

3. Difference between \n and \t

\n	\t
New line / line feed: inserts a new line and the cursor moves to the beginning of the next line	Horizontal tab: shifts the cursor a couple of spaces to the right in the same line

Operators in C++

1. Arithmetic operators vs relational operators

Arithmetic operators	Relational operators
(+, -, *, /, %) perform arithmetic operations / calculations on operands and give a numeric result	(==, !=, >, <, >=, <=) test the relation between two values; they are binary operators, they need two operands, and their result is True (1) or False (0)

The **!=** operator belongs to the relational operators — it means "not equal to" and compares the values of two operands. To compare two values, <= and == are both correct comparison operators; = is the assignment operator, not a comparison.

2. Logical operators

The operator used for the logical AND operation is **&&** — the condition will be true if both expressions are true. The logical operators are && (AND), || (OR) and ! (NOT).

3. Assignment operator vs equal to operator

The **assignment operator (=)** assigns the value of the right-side expression to the left-side variable, for example `x=10;` The **equal to operator (==)** checks the equality of the values of two operands, for example `x==y;` — if the condition is true the result is true, otherwise false. The **+=** operator adds the right operand to the left operand and assigns the result to the left operand.

4. Output of a program with a++, --a

Program: `int a = 27;` then a is displayed with a++, a, --a and a. Output: a is 27; a is now 27; a is now 28; a is now 27; a is now 27. Reason: a++ is postfix, first 27 is printed then a becomes 28; then a is printed as 28; --a is prefix, first a becomes 27 then 27 is printed; finally a is printed as 27.

5. Operator precedence: a worked example

`cout << 12-6/2;` gives **9** on the screen: by the precedence of arithmetic operators, division comes first: 6/2 = 3, then 12-3 = 9.

Programs: Formulas, Marksheet and Finding Errors

1. Program using all arithmetic assignment operators

```
#include<iostream> using namespace std; int main() { int a = 20; a += 2; cout << "After += 2, a = " << a << endl; a -= 3; cout << "After -= 3, a = " << a << endl; a *= 4; cout << "After *= 4, a = " << a << endl; a /= 4; cout << "After /= 4, a = " << a << endl; return 0; }
```

Output: After += 2, a = 22; After -= 3, a = 19; After *= 4, a = 76; After /= 4, a = 19.

2. Programs for formulas

- **Speed** s = d/t: input d and t; s = d / t; display s.
- **Newton's second law** F = ma: input m and a; F = m * a; display F.
- **Acceleration** a = (vf - vi)/t: input vf, vi and t; a = (vf - vi) / t; display a.
- **Area of triangle** a = 1/2 bh: input b and h; area = (b * h) / 2; display area.
- **Celsius to Fahrenheit** F = (c * 1.8) + 32: input c; F = (c * 1.8) + 32; display F.

Every program has the same structure: #include<iostream>, using namespace std;, int main(), declaration of variables, input with cin, formula, result with cout, return 0;

3. Program: volume of a box

```
#include<iostream> using namespace std; int main() { float length, width, height, volume; cout << "Enter length, width and height"; cin >> length >> width >> height; volume = length * width * height; cout << "Volume = " << volume; return 0; }
```

Volume = length x width x height (the formula of Algorithm 4 of Unit 1).

4. Program: marksheet of five subjects

```
#include<iostream> using namespace std; int main() { float m1, m2, m3, m4, m5, total, percentage; cout << "Enter marks of five subjects"; cin >> m1 >> m2 >> m3 >> m4 >> m5; total = m1 + m2 + m3 + m4 + m5; percentage = total / 500 * 100; cout << "Total = " << total << " Percentage = " << percentage; return 0; }
```

Total 500 because each subject is of 100 marks (like the book's three-number model: Percent = Sum / 300 * 100).

5. Program: the expression a2 + 2ab + b2

```
#include<iostream> using namespace std; int main() { float a, b, result; cout << "Enter a and b"; cin >> a >> b; result = a * a + 2 * a * b + b * b; cout << "Result = " << result; return 0; }
```

This expression equals the square of (a + b).

6. Finding and removing errors from a program

Errors: 1. `using namespace std` is missing a semicolon (;) at the end; 2. `int main()` must not be followed by (;); 3. `cin >> x` is missing (;) at the end; 4. in a * a, a is not declared — it should be x * x. Correct program:

```
#include<iostream> using namespace std; int main() { int x; cout << "Enter the value of x"; cin >> x; cout << "The square of x is " << x * x; return 0; }
```

Output (for input 5): Enter the value of x 5; The square of x is 25.

Control Structure

UNIT 4

Selection: if, else if and switch

1. The if statement and the use of else

Using **else** is **optional** with the if statement — if can be used alone; with else, in if-else either the if block executes or the else block. The if and loop expressions use **relational operators** to test the condition.

2. The else if ladder

else if tests many conditions in a ladder — the block of whichever condition is true runs, otherwise the next condition is tested; it makes a nested-if program simpler and more readable.

3. The switch statement

switch matches the variable's value with the case constants and transfers control to the matching case; if none matches, to **default**. "case" and "default" are parts of the switch statement.

4. Purpose of default

In a switch, if the switch variable does not match any of the case constants, control goes to default (if it is present). Default acts as "else".

5. Which data types can be used in switch?

In a switch, only **character and integer** data type variables can be used; it matches the switch variable's value only with case constants, not with any variable, and relational operators like < or > cannot be used in it — the value must exactly match the case constants.

Loops: for, while and do-while

1. The for loop

The for loop executes a sequence of statements multiple times. It is usually used in situations where we know at the start of the loop how many times the loop block will execute; it is a counter-controlled / definite repetition loop and the condition is tested at the start of the loop. The for loop expression has **three parts**: first initialization, second condition testing, third increment or decrement.

2. while vs do-while

while loop	do-while loop
Pre-test loop: the condition is tested at the start of the loop, so the body may not run even once	Post-test loop: the condition is tested at the end of the body, so its body runs at least once; its while(condition) is followed by a semicolon

3. Using while in place of for

Yes. The three parts of a for loop can be split up: initialization before the loop, the condition in while(...), and increment / decrement inside the body of the loop. For example: `int i = 1; while (i <= 10) { cout << i; i++; }` — this is the same as for (i = 1; i <= 10; i++).

4. Nested loops

A loop within a loop is called a **nested loop** — if a loop exists in the body of another loop, then it is called a nested loop.

Jump Statements and Blocks of Statements

1. Purpose of jump statements

Jump statements change the execution of the program from its normal sequence: they transfer control to another part of the program, terminate a loop or switch, send a value back from a function, or terminate the program.

2. break and continue

- **break** is used with switch (and loops) — break terminates the loop or switch statement; in a switch, break comes at the end of each case.
- **continue** takes control to the **top of the loop** — it causes the loop to skip the remaining statements of its body and immediately transfers control to the top.

3. goto, return and exit()

- **goto:** jumps control unconditionally from the goto to a labeled statement in the same function; in the goto statement the label is followed by a **colon (:)** character — a labeled statement is any identifier followed by a colon (:).
- **return:** to send a value to the calling function we use the return statement; it terminates the execution of a function and can also return a value from the function.
- **exit():** the exit() function is used to close the program — it terminates the C++ program and closes it whenever executed.

4. Why we make blocks of statements using braces

To gather more than one statement into one group / block, they are enclosed in curly braces, so that they run together as one joint part; for example, if there is more than one statement in an if block, the body of a loop or the body of a function, they are all enclosed in braces.

5. Matching pairs to remember

Item	Matches with
if	else — if and else together make the if-else pair
loop	iteration — a loop means running statements again and again, i.e. iteration
Conditional expression	Relational operators — conditional expressions are tested with relational operators
Loops and switch	break — break terminates the loop or switch statement
do	while — do and while together make the do-while loop
>>	operator — >> is an operator (the extraction / input operator)
case	switch — case is a part of the switch statement

Programs with Selection and Loops

1. Program: odd or even

```
#include<iostream> using namespace std; int main() { int num; cout << "Enter a number"; cin >> num; if (num % 2 == 0) cout << "Even"; else cout << "Odd"; return 0; }
```

If dividing the number by 2 leaves remainder 0, it is even, otherwise odd.

2. Program: add numbers from 1 to 20

```
#include<iostream> using namespace std; int main() { int i, sum = 0; for (i = 1; i <= 20; i++) sum = sum + i; cout << "Sum = " << sum; return 0; }
```

Output: Sum = 210.

3. Program: number of days in a month (switch)

```
#include<iostream> using namespace std; int main() { int month; cout << "Enter month number 1 to 12"; cin >> month; switch (month) { case 1: case 3: case 5: case 7: case 8: case 10: case 12: cout << "31 days"; break; case 4: case 6: case 9: case 11: cout << "30 days"; break; case 2: cout << "28 days"; break; default: cout << "Wrong number / Error message"; } return 0; }
```

February is written as 28 days; the book does not give the leap-year case.

4. Program: each digit on a separate line

```
#include<iostream> using namespace std; int main() { int num, divisor = 1, temp; cout << "Enter a number up to six digits"; cin >> num; temp = num; while (temp >= 10) { divisor = divisor * 10; temp = temp / 10; } while (divisor > 0) { cout << num / divisor << endl; num = num % divisor; divisor = divisor / 10; } return 0; }
```

The first loop finds the divisor of the largest place, the second loop prints each digit from left to right on a separate line.

5. Program: square box filled with a character (nested loops)

```
#include<iostream> using namespace std; int main() { char ch; int rows, cols, i, j; cout << "Enter character, rows and columns"; cin >> ch >> rows >> cols; for (i = 1; i <= rows; i++) { for (j = 1; j <= cols; j++) cout << ch; cout << endl; } return 0; }
```

The outer loop runs the rows, the inner loop prints the character in each row.

6. Programs: number patterns with nested loops

(1) Stars (* / ** / *** / **** / *****): `for (i = 1; i <= 5; i++) { for (j = 1; j <= i; j++) cout << "*"; cout << endl; }`

(2) Numbers (1 / 12 / 123 / 1234 / 12345): `for (i = 1; i <= 5; i++) { for (j = 1; j <= i; j++) cout << j; cout << endl; }`

(3) Same number (1 / 22 / 333 / 4444 / 55555): `for (i = 1; i <= 5; i++) { for (j = 1; j <= i; j++) cout << i; cout << endl; }`

All three are made with nested loops — the outer loop runs the row, the inner loop runs the items of each row.

7. Program: prime or not

```
#include<iostream> using namespace std; int main() { int n, i; bool isPrime = true; cout << "Enter a number"; cin >> n; if (n <= 1) isPrime = false; for (i = 2; i <= n / 2; i++) { if (n % i == 0) isPrime = false; } if (isPrime) cout << "Prime"; else cout << "Not Prime"; return 0; }
```

A prime number is one that is divided only by 1 and itself.

8. Program: designation from salary (else if ladder)

```
#include<iostream> using namespace std; int main() { float salary; cout << "Enter salary"; cin >> salary; if (salary >= 100000) cout << "Manager"; else if (salary >= 50000) cout << "Supervisor"; else cout << "Worker"; return 0; }
```

The book does not fix salary limits; the limits above are given as an example.

9. Program: squares of numbers from 1 to 10

```
#include<iostream> using namespace std; int main() { int i; for (i = 1; i <= 10; i++) cout << i << " square = " << i * i << endl; return 0; }
```

Output: 1 square = 1, 2 square = 4, 3 square = 9, 4 square = 16, 5 square = 25, 6 square = 36, 7 square = 49, 8 square = 64, 9 square = 81, 10 square = 100.

10. Program: table of a number using while loop

```
#include<iostream> using namespace std; int main() { int n, i = 1; cout << "Enter a number"; cin >> n; while (i <= 10) { cout << n << " x " << i << " = " << n * i << endl; i++; } return 0; }
```

For example, for 2 the output is: 2 x 1 = 2, 2 x 2 = 4, 2 x 3 = 6, ..., 2 x 10 = 20.

Functions

UNIT 5

Functions: Built-in and User-defined; Declaration and Definition

1. Two kinds of functions

The functions that are defined by the programmer are called **user-defined functions** — the programmer can write their own functions to perform a specific task. The functions already given by C++ are the **built-in (predefined) functions**.

2. Function prototype (declaration)

The word "prototype" means **declaration**. The function prototype consists of all of these: the name of the function, the parameters passed to the function, and the value returned from the function — a function declaration tells the compiler about the function's name, return data type and arguments/parameter data types.

3. Function declaration vs function definition

Declaration	Definition
A function without its definition (code block); declared before main()	The function itself; it has a header and a body / code block
Tells the compiler about the function's name, return data type, and arguments/parameter data types; ends with a semicolon (;)	The header has the return value data type, the function name and the list of arguments with datatypes in parenthesis; the body has statements in braces; the definition may be before or after the main function

4. The void keyword

void is used when no value is returned by the function. If there is no parameter in the function, void is also used, for example `void function name(void);`

5. Including a function through a header file

A programmer creates a function for a particular task and includes that function in the program through a header file, which has the **.h** extension — predefined functions are declared in header files and included in the program with #include.

6. main() returns int

In C++, `int main()` returns an **int** (integer) data type value by default to the operating system.

Header Files, Built-in Functions and Variable Scope

1. Why we use header files

Predefined functions are declared in header files; to use them the proper header file is included with #include, for example cmath for sqrt(), cstdio for gets() / puts(), conio.h for getch() / getche(). Header files link the external header files / libraries which may be required in the program.

2. Five standard built-in functions

- **sqrt(a):** finds the square root of a number.
- **pow(x, y):** finds x to the power y.
- **strcpy(str1, str2):** copies one string into another string.
- **toupper(ch):** converts a character to upper case.
- **getch():** gets a single character from the keyboard.

3. Parameters: formal and actual

The parameters specified in the function header are called **formal parameters**. All variables declared in a function definition are called **local variables**.

4. Local variables vs external (global) variables

External / global variables	Local variables
Declared outside of the main function; also known as external variables	Declared inside any function (also in the header of the function definition)
Hold the value of the variable during the entire execution of the program; their value can be shared with different functions	Only accessible within a specific part of the program / that function

5. Advantages of user-defined functions

- Large and complicated programs are broken down into smaller modules / functions.
- Functions help the programmer to manage the code of the large program.
- A function written once can be called from any part of the program.
- The programmer can create functions for a specific task.

Passing Arguments and Returning Values

1. Passing an argument to a function

When the function is called, actual values (variables or constants) are given with the function call; they are received in the **formal parameters** of the function definition, which act as local variables inside the function.

2. Returning a value from a function

When the function completes its execution, the **return** keyword sends a single value back to the calling function; the return data type must be specified in the header, before the function name, in both the function declaration and the definition.

3. Finding the error and the output of a program

Error: in the function prototype line, `\\` is written, which is not a comment in C++; a comment needs `//`: `void Table(void); // Function prototype`. There is no other error. Output (for input m = 5): The value of m..... 5; then the table of 5: 5*1=5, 5*2=10, 5*3=15, 5*4=20, 5*5=25, 5*6=30, 5*7=35, 5*8=40, 5*9=45, 5*10=50.

Programs Using Functions

1. Series program — no return value and no parameters

```
#include<iostream> using namespace std; void series(void); int main() { series(); return 0; } void series(void) { int i; for (i = 0; i <= 35; i = i + 5) cout << i << " "; }
```

Output: 0 5 10 15 20 25 30 35. The function neither returns a value nor takes parameters: void function name(void);

2. Even or odd — return value and pass parameters

```
#include<iostream> using namespace std; int checkEven(int n); int main() { int num, result; cout << "Enter a number"; cin >> num; result = checkEven(num); if (result == 1) cout << "Even"; else cout << "Odd"; return 0; } int checkEven(int n) { if (n % 2 == 0) return 1; else return 0; }
```

The function takes a parameter and returns a value: int function name(int);

3. Convert kilograms into grams using a function

```
#include<iostream> using namespace std; float kgToGrams(float kg); int main() { float kg, grams; cout << "Enter kilograms"; cin >> kg; grams = kgToGrams(kg); cout << "Grams = " << grams; return 0; } float kgToGrams(float kg) { return kg * 1000; }
```

The function takes the value in kilograms as a parameter and returns the value in grams.

4. Box of stars function

```
#include<iostream> using namespace std; int starBox(int length, int height); int main() { int total; total = starBox(10, 4); cout << "Total stars = " << total; return 0; } int starBox(int length, int height) { int i, j; for (i = 1; i <= height; i++) { for (j = 1; j <= length; j++) cout << "*"; cout << endl; } return length * height; }
```

Output: 4 rows of 10 stars, then Total stars = 40.

5. Factorial function

```
#include<iostream> using namespace std; int factorial(int n); int main() { int num; cout << "Enter a number"; cin >> num; cout << "Factorial = " << factorial(num); return 0; } int factorial(int n) { int i, f = 1; for (i = 1; i <= n; i++) f = f * i; return f; }
```

Factorial means n! = 1 x 2 x ... x n.

6. Formula functions

- **Area of rectangle** a = l x w: `float rectArea(float l, float w) { return l * w; }`
- **Area of circle** a = pi r2 (pi = 3.14): `float circleArea(float r) { return 3.14 * r * r; }`
- **Pythagorean theorem** c2 = a2 + b2: `float hypotenuse(float a, float b) { return sqrt(a * a + b * b); }` — from c2 = a2 + b2, c = sqrt(a2

+ b2).

Each function is called from main() with parameters and returns the result.

Digital Logic and Design

UNIT 6

Basic Logic Gates: AND, OR and NOT

1. The AND gate

The AND gate has two or more inputs and a single output; it operates on logical multiplication; **Y = A.B**; the output is 1 only when all inputs are 1 — if any input is 0 the output is 0.

A	B	Y = A.B
0	0	0
0	1	0
1	0	0
1	1	1

2. The OR gate

The OR gate has two or more inputs and a single output; it operates on logical addition; **Y = A+B**; the output is 1 if any input is 1 — if all inputs are 0 the output is 0.

A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

3. The NOT gate — the inverter

The NOT gate has a single input and a single output; it is the **inverter**; **Y = $\bar{A}$** (A with a bar over it). The bar sign over a variable indicates the NOT operation. The inverter inverts the input: if the input is 1 the output is 0, if the input is 0 the output is 1.

A	Y = $\bar{A}$
0	1
1	0

Universal Gates: NAND and NOR

1. Universal gates

The **NAND gate** and the NOR gate are the universal gates.

2. The NAND gate

NAND gate (NOT-AND) is the collection of an AND gate and a NOT gate connected in series; **Y = (A.B) with a bar over it**; the output is 0 only when all inputs are 1 — if any one of the inputs is 0 the output is 1. So, when A and B go into an AND gate and its output passes through a NOT gate, the Boolean expression of the circuit is (A.B) with a bar over it — the circuit is a NAND gate.

3. The NOR gate

The NOR gate is the collection of an OR gate and a NOT gate; its output is inverted OR; **Y = (A+B) with a bar over it**; the output is 1 only when both inputs are 0 — if any one of the inputs is 1 the output is 0.

4. NAND vs NOR

NAND gate	NOR gate
AND gate + NOT gate in series; Y = (A.B) with a bar over it	OR gate + NOT gate; Y = (A+B) with a bar over it
Output is 0 only when all inputs are 1	Output is 1 only when both inputs are 0
Universal gate	Universal gate

Boolean Algebra and Truth Tables

1. Why we use Boolean algebra

Boolean algebra is used to describe the logical operations and processing of binary information; its pre-defined rules help to **simplify the logical expression**, so that the number of logic gates needed to perform a particular logic operation is reduced.

2. Truth tables

A truth table is a tabular representation of all the combinations of values for inputs and their corresponding outputs. With the combination of three variables, **eight** outputs are expected altogether — for three variables the truth table has 2 x 2 x 2 = 8 combinations / rows.

3. Rules of Boolean algebra used in this unit

Rule	Statement
Rule 2	1 + A = 1
Rule 4	A . 1 = A
Rule 5	A + A = A
Rule 7	A . A = A (for example B . B = B)
Rule 8	A . $\bar{A}$ = 0
Rule 10	A + A.B = A
Rule 11	$\bar{A}$ + A.B = $\bar{A}$ + B (for example $\bar{B}$ + B.C = $\bar{B}$ + C)

So A+A=A is the 5th rule of Boolean algebra, and A.$\bar{A}$=0 is the 8th rule.

Simplifying Expressions and Designing Circuits

1. Simplify: Z = AB + A(B + C) + B(B + C)

Expand: AB + AB + AC + BB + BC = AB + AC + B + BC (because BB = B, Rule 7, and AB + AB = AB, Rule 5). B + BC = B (Rule 10: A + A.B = A). Then Z = AB + AC + B = B + AB + AC = B(1 + A) + AC = B + AC (Rule 2: 1 + A = 1; Rule 4: B.1 = B). So **Z = B + AC**.

2. Simplify: two short expressions

- **ABC + AB$\hat{C}$ + $\bar{A}$B = B** — take AB common from the first two parts: AB(C + $\hat{C}$) = AB(1) = AB; then AB + $\bar{A}$B = B(A + $\bar{A}$) = B(1) = B.
- **ABC + $\bar{A}$BC = BC** — take BC common: BC(A + $\bar{A}$) = BC(1) = BC.

3. Logic circuit of the 10th law of Boolean algebra

10th Rule: A + A.B = A. Circuit: A and B go into an AND gate to give A.B; then A and A.B go into an OR gate to give A + A.B, whose output is A.

4. Designing a circuit from a Boolean expression

Expression: Q = (A.B) + (A.B) + (A+B) with a bar over (A+B). Circuit: two AND gates (both with A and B) give the two A.B parts; one OR gate (with A and B) followed by a NOT gate gives (A+B) with a bar over it; then the three parts go into a three-input OR gate, whose output is Q. Note: A.B is printed twice in the expression; AB + AB = AB (Rule 5), so Q = AB + (A+B) with a bar over (A+B).

5. From a circuit to the expression, truth table and simplification

Circuit: A, B, C into a three-input AND gate; B and C through NOT gates into an OR gate; A and ($\bar{B}$ + C) into an AND gate; both results into a final OR gate. Expression: **Y = ABC + A($\bar{B}$ + C)**.

A	B	C	ABC	$\bar{B}$+C	A($\bar{B}$+C)	Y
0	0	0	0	1	0	0
0	0	1	0	1	0	0
0	1	0	0	1	0	0
0	1	1	0	0	0	0
1	0	0	0	1	1	1
1	0	1	0	1	1	1
1	1	0	0	1	1	1
1	1	1	1	0	0	1

Simplify: Y = A(BC + $\bar{B}$ + C); $\bar{B}$ + BC = $\bar{B}$ + C (Rule 11); so Y = A($\bar{B}$ + C + C) = A($\bar{B}$ + 1) = A (Rule 6 and Rule 2 and Rule 4). So the simplified form is: **Y = A**.

Introduction to Scratch

UNIT 7

Scratch Basics: Sprites, Scripts and Backdrops

1. Features of Scratch

- It is a **visual** language.
- It is **free forever**.
- There is **no need to remember codes**.

All of these are features of Scratch.

2. Sprite, script and backdrop

- **Script:** the set of step-wise instructions given to the sprite to do a particular task; scripts are instructions that make sprites perform a task; each sprite has its own script area.
- **Sprite:** the images of cartoons, characters or objects added in the project — the character which moves on the Stage is called a sprite. We can have multiple sprites in a project but at least one sprite is always needed; Cat is the default sprite.
- **Backdrop:** the background added on the stage; by default, no backdrop is added in a project; the look of the stage can be changed by adding new backdrops.

3. What a Scratch program is made of

A Scratch program is at least made up of **one sprite and code** — at least one sprite is always needed for the project, and every sprite has its own code / script.

The Stage, Coordinates and Block Palettes

1. Coordinates on the stage

To change the position of a sprite on the screen, we use **coordinates** — the stage is divided into x (horizontal) and y (vertical) coordinates; the go to x, y command changes the position of the sprite. Example: Point A in the book's picture is at **x=100, y=-100** — Point A is in the lower-right quadrant, where x is positive and y is negative.

2. Marking points on the 480 x 360 stage matrix

Point	Position
(a) x=-170, y=100	Upper-left (second) quadrant — x negative, y positive
(b) x=110, y=190	Upper-right (first) quadrant — x positive, y positive (the stage's y range is -180 to 180, so y=190 lies slightly outside the top edge)
(c) x=-120, y=-120	Lower-left (third) quadrant — x negative, y negative
(d) x=150, y=0	On the x-axis (the middle horizontal line), on the right side

To check the marks: for each point use the go to x: ... y: ... block and see whether the sprite goes to the marked place or not.

3. Block palettes and their colours

Palette	Colour
Motion	Blue
Events	Brown / orange
Sound	Pink
Looks	Purple
Control	Golden yellow
Sensing	Light blue
Data	Orange
Pen	Green
Operators	Green

Each block group has its own associated colour. The colours differentiate the different commands and help to quickly find the right block / option — the book says the colours will help you to find the option.

4. Changing the look of a sprite

The looks of a sprite can be changed by using the **Costume Tab** — the costume of a sprite can be changed by clicking on the "Costumes" tab.

Motion and Control Blocks

1. Motion blocks

- **move 10 steps** is found under **Motion** — it moves the sprite in its current direction for the given steps.
- The **turn** command turns the sprite to the specified **degree** — the turn 15 degrees command turns the sprite to the specified degree.
- **go to x, y** sends the sprite to the specified x, y coordinates (position on stage).

2. repeat 10 and forever

repeat 10, forever and if ... then codes are available in the **Control** group. **repeat 10** repeats the following code blocks for the specified number of times (10 times); **forever** keeps repeating the following code blocks for infinite times.

3. Use of common blocks

Block	Use
forever	Keeps repeating the following code blocks for infinite times
wait	Waits for specified seconds
say	Says the given words / phrase (say ... for 2 secs says it for the specified seconds)
play sound	Plays the specified sound
go to x, y	Sends the sprite to the specified x, y coordinates (position on stage)

Scratch Online and Offline; Sample Programs

1. Scratch online vs offline

Online	Offline
The editor is loaded in the web browser at scratch.mit.edu/projects/editor and used; it functions just like the offline editor; an account can be created on Scratch to save projects online	Scratch is downloaded and installed from the MIT website on a computer or an Android device to work offline (run the exe file, choose the location, shortcuts)

2. Sample program: two sprites talk with each other

When flag clicked: Sprite 1: say "Hello! How are you?" for 2 secs; Sprite 2: say "I am fine! What about you?" for 2 secs; Sprite 1: say "I am fine too!" for 2 secs; Sprite 2: say "Why did the computer go to school? To improve its memory!" for 2 secs. Each sprite has its own code block and the turn changes with a wait block between the say blocks. (The conversation lines are a sample.)

3. Program: area and circumference of a circle

When flag clicked: ask "Enter radius" and wait; set radius to answer; set area to 3.14 x radius x radius; set circumference to 2 x 3.14 x radius; say (join "Area = " area); say (join "Circumference = " circumference).